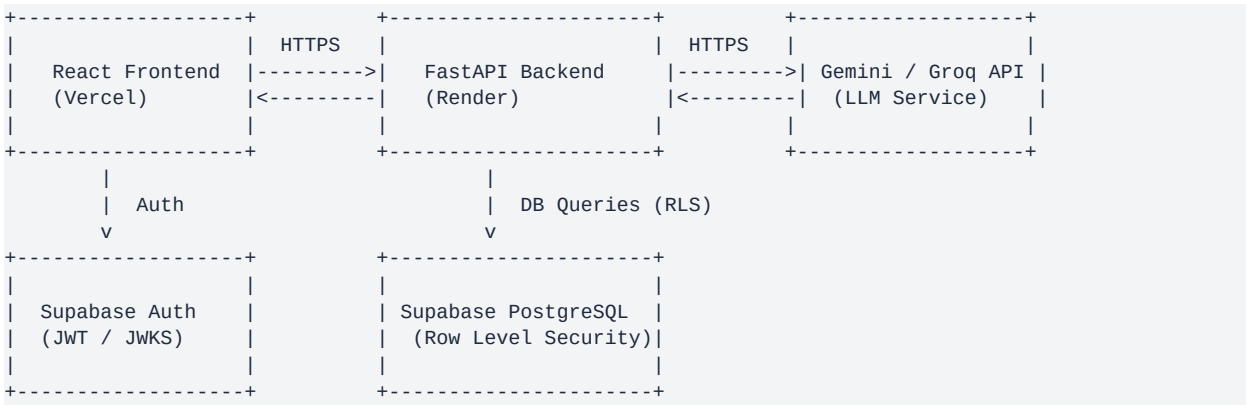
Solution Architecture

Date	21 October 2026
Team ID	TDS-AI-2026
Project Name	Personalized Travel Itinerary Planner
Maximum Marks	5 Marks

Project Title

Personalized Travel Itinerary Planner

High-Level Architecture



Component Architecture

Frontend Architecture (React)

```
App.tsx
■■■ AuthProvider (Supabase session management)
■   ■■■ LoginPage (email/password auth)
■   ■■■ SignupPage (new user registration)
■   ■■■ ProtectedRoute (auth guard)
■       ■■■ AppShell (layout with navigation)
■           ■■■ DashboardPage (trip list + delete)
■           ■■■ NewTripPage (trip form + AI generation)
■           ■■■ TripPage (day-by-day itinerary view)
■               ■■■ Day selector buttons
■               ■■■ Day regeneration
■               ■■■ DndContext (drag-and-drop)
■               ■   ■■■ SortableContext
■               ■       ■■■ SortableCard (activity cards)
■               ■■■ ActivityEditDialog (edit modal)
```

Backend Architecture (FastAPI)

```
app/
■■■ main.py              (FastAPI app, CORS, middleware, routers)
■■■ core/
■   ■■■ config.py        (Settings from .env via Pydantic)
■   ■■■ security.py      (JWT verification via JWKS)
■   ■■■ supabase.py      (Per-user Supabase client factory)
```

```
■   ■■■ errors.py        (Centralized exception handlers)
■■■ routers/
■   ■■■ health.py        (GET /api/health)
■   ■■■ me.py            (GET /api/me — user profile)
■   ■■■ trips.py         (CRUD + generate + regenerate + reorder)
■   ■■■ activities.py    (PATCH + DELETE individual activities)
■■■ schemas/
■   ■■■ trip.py          (TripCreate, TripUpdate, TripOut, TripDetailOut)
■   ■■■ day.py           (DayOut)
■   ■■■ activity.py      (ActivityOut, ActivityUpdate)
■■■ services/
■   ■■■ trip_service.py  (Business logic: CRUD, cost computation)
■   ■■■ generation_service.py (Trip + day generation orchestration)
■   ■■■ ai_service.py    (LLM calls, prompt building, response validation)
■■■ repositories/
■   ■■■ trip_repo.py     (Raw Supabase queries)
■■■ prompts/             (Reserved for prompt template files)
```

Database Architecture (Supabase PostgreSQL)

```
auth.users (managed by Supabase)
    |
    v (trigger: handle_new_user)
profiles -----> trips -----> itinerary_days -----> activities
                 |
                 +-----> trip_members
                 +-----> generation_logs
```

Security Architecture

Layer	Mechanism
Authentication	Supabase Auth (email/password) → JWT tokens
Token Verification	JWKS-based verification with key caching (600s lifespan)
API Authorization	FastAPI dependency injection (get_current_user)
Data Authorization	Row-Level Security (RLS) with SECURITY DEFINER helper functions
Client Scoping	Per-request Supabase client using the user's access token
CORS	Whitelisted frontend origin only
Input Validation	Frontend: Zod schemas; Backend: Pydantic models

API Endpoints Architecture

Method	Endpoint	Description
GET	/api/health	Health check (no auth required)
GET	/api/me	Get current user profile
POST	/api/trips	Create trip (manual, no AI)
GET	/api/trips	List all user's trips
GET	/api/trips/{id}	Get trip with full itinerary

Method	Endpoint	Description
PATCH	/api/trips/{id}	Update trip metadata
DELETE	/api/trips/{id}	Delete trip
POST	/api/trips/generate	Create trip with AI-generated itinerary
POST	/api/trips/{id}/days/{n}/regenerate	Regenerate single day
POST	/api/trips/{id}/days/{n}/reorder	Reorder activities in a day
PATCH	/api/activities/{id}	Update single activity
DELETE	/api/activities/{id}	Delete single activity